

Multi-layer Abstraction for Nested Generation of Options (MANGO) in Hierarchical Reinforcement Learning

Alessio Arcudi^{#*} Davide Sartor^{#**} Alberto Sinigaglia^{*}
Vincent François-Lavet^{***} Gian Antonio Susto^{*,**}

^{*} *Human Inspired Technology Research Center, Università di Padova,
Padova, PD 35121 IT.*

^{**} *Dep. of Information Engineering, Università di Padova, Padova,
PD 35121 IT.*

^{***} *Vrije Universiteit Amsterdam, Amsterdam, Netherlands*

Abstract: This paper introduces MANGO (Multilayer Abstraction for Nested Generation of Options), a novel hierarchical reinforcement learning framework designed to address the challenges of long-term sparse reward environments. MANGO decomposes complex tasks into multiple layers of abstraction, where each layer defines an abstract state space and employs options to modularize trajectories into macro-actions. These options are nested across layers, allowing for efficient reuse of learned movements and improved sample efficiency. The framework introduces intra-layer policies that guide the agent’s transitions within the abstract state space, and task actions that integrate task-specific components such as reward functions. Experiments conducted in procedurally-generated grid environments demonstrate substantial improvements in both sample efficiency and generalization capabilities compared to standard RL methods. MANGO also enhances interpretability by making the agent’s decision-making process transparent across layers, which is particularly valuable in safety-critical and industrial applications. Future work will explore automated discovery of abstractions and abstract actions, adaptation to continuous or fuzzy environments, and more robust multi-layer training strategies.

Keywords: Hierarchical Reinforcement Learning, Deep Reinforcement Learning, Machine Learning, Neural Network.

1. INTRODUCTION

Reinforcement Learning (RL) has proved immensely successful in tackling a variety of decision-making tasks, yet its efficacy can diminish in environments characterized by large state spaces and sparse rewards. As the number of possible states expands and the temporal horizon grows, standard RL methods often struggle to identify effective strategies. This challenge has spurred the development of Hierarchical RL (HRL) approaches, which divide complex tasks into smaller, more manageable subtasks (Dietterich, 1999; Sutton et al., 1999; Hengst et al., 2002).

Hierarchy in artificial intelligence has been studied for decades (Sacerdoti, 1974). A central concept in HRL involves decomposing an overarching Markov Decision Process (MDP) into multiple, smaller MDPs (Sutton et al., 1999; Silver and Ciosek, 2012; Konidaris and Barto, 2009). These sub-MDPs capture subtasks defined by core states or transitions of interest. Ho et al. (2019, 2021) further clarify hierarchy in RL by distinguishing between temporal abstractions, spotlighting critical states in subtrajectories, and state-action abstractions, which compress the original

state space and represent subtrajectories as transitions in an abstract domain. Building on state-action abstraction, this work aligns with ideas proposed by Dietterich (1999); Ravindran and Barto (2003); Abel et al. (2020), as well as the Reward Machines paradigm (Icarte et al., 2022).

This work introduces a novel hierarchical, multi-layer framework named MANGO. Its principal aim is to learn effectively in complex, sparse-reward environments by exploiting a structured decomposition of the environment. In addition to improving learning efficiency, a key objective is to enhance the intrinsic interpretability of the model. By decomposing the agent’s decision-making process across multiple levels, the user can track which subtask or higher-level goal influences the agent’s actions at each timestep. This level of transparency can be invaluable in industrial or safety-critical domains, where understanding an agent’s rationale and, when not aligned with the objective, correcting it is as important as its overall performance.

2. BACKGROUND

The concept of planning in a hierarchy of abstraction states has always been a topic of interest in the field of artificial intelligence Sacerdoti (1974). This approach to planning involves breaking down a complex task into a series of smaller, more manageable subtasks, which are

[#] Equal contribution. Corresponding author: Davide Sartor (email: davide.sartor.4@phd.unipd.it). Authors’ email: alberto.sinigaglia, alessio.arcudi@phd.unipd.it, vincent.francoislavet@vu.nl, gianantonio.susto@unipd.it.

then planned and executed independently at different levels of abstraction.

The most commonly used family of hierarchical reinforcement learning methods for this approach is the temporal difference methods. These methods involve breaking down a task into temporal lags that define specific subgoals, which can then be grouped to form a macro subgoal. This allows the agent to learn to identify and achieve intermediate objectives that help to achieve the main objective, such as in Sutton et al. (1999), Silver and Ciosek (2012), and Konidaris and Barto (2009).

In Abel (2022), a review of the literature is presented, highlighting the main differences and characteristics of the two approaches. In recent years, significant research has been carried out on these topics, as demonstrated in Rafati and Noelle (2018), Konidaris and Barto (2009), and Konidaris et al. (2015). However, it is worth noting that the two types of abstraction are not entirely distinct, and there are cases where they interact in state-action abstraction, as described in Dietterich (1999), Ravindran and Barto (2003), and Abel et al. (2020).

The primary concept presented in these papers involves decomposing the Markov Decision Process (MDP) into a hierarchical structure of smaller MDPs, which are defined by specific state aggregations utilizing state abstraction. These MDPs subsequently define subtasks of the original MDP. In the work Abel et al. (2020), a mathematical formulation of state and action abstractions is provided, wherein they introduce a novel method for combining state abstractions with abstract actions, also called options Sutton et al. (1999), by grouping them through a bisimulation metric.

We present a framework for hierarchical reinforcement learning that builds on the work of Abel et al. (2020) and Dietterich (1999). Our framework uses functions that can synthesize and abstract features of the environment in a hierarchical fashion. Specifically, each layer of the hierarchy refers to a specific feature-state space, and the framework is able to discover and refine options that are defined as abstract state modifications in a particular layer of the state space. To facilitate the learning process, we first perform a prior task-independent discovery of the options of the environment, as suggested in Veeriah et al. (2021). We then define a reinforcement learning environment where the agent learns to perform options (macro-actions on the environment) rather than atomic actions.

In summary, our framework provides a hierarchical approach to reinforcement learning that allows for the synthesis and abstraction of environment features, discovery of options, and efficient learning through the use of macro-actions.

3. MATHEMATICAL FRAMEWORK

3.1 Preliminaries

A Markov Decision Process (MDP) is a mathematical framework for modeling decision-making in stochastic environments. An MDP is formally defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{T}, R, \gamma)$, consisting of a state space $\mathcal{S}$, a action space

$\mathcal{A}$, a transition function $\mathcal{T}: \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{P}(\mathcal{S})$, a reward function $R: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ and a discount factor $\gamma \in [0, 1)$.

An MDP captures how an agent interacts with the environment over time: at each step, the agent observes a state $s \in \mathcal{S}$, takes an action $a \in \mathcal{A}$, transitions to a new state $s' \sim \mathcal{T}(s, a)$, and receives a reward $r = R(s, a, s')$. To conclude, the goal is to learn a policy $\pi(a|s)$ which aims at maximizing the expected return, i.e. discounted sum of rewards J :

$$J = \mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right]. \quad (1)$$

Options, introduced by Sutton et al. (1999), provide a structured mechanism for integrating abstract behaviors in the agent’s decision process. Options are learned sub-routines that can be executed in the environment, and can be used by the agent in an analogous way to actions, essentially expanding the action space of the agent.

Traditionally, options consist of a policy guiding agents within states, an initialization condition defining in which states the option can be initiated, and a termination condition defining when the option ends. In our framework, we simplify the definition, omitting the initialization condition, assuming that options can be initiated in any state of the environment.

Definition 3.1. (Options). Options consist of two components $o: = \langle \pi_o, \beta_o \rangle$:

- a policy $\pi_o: \mathcal{S} \rightarrow \mathbb{P}(\mathcal{A})$ guiding actions within states,
- a condition $\beta_o: \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \{False, True\}$ defining if the option should terminate after a transition.

The *option space* is defined as the set of available options:

$$\mathcal{O} = \{o_1, o_2, \dots, o_k\},$$

where each o_i is an option as previously defined. This expands the action space of the agent from primitive actions $\mathcal{A}$ to a combined action-option space $\mathcal{A}' = \mathcal{A} \cup \mathcal{O}$.

Consequently, the agent’s policy π becomes hierarchical, mapping states to distributions over this expanded space:

$$\mu: \mathcal{S} \rightarrow \mathbb{P}(\mathcal{A}'), \quad (2)$$

thus enabling the agent to choose either a primitive action or initiate an option at each step. When an option $o \in \mathcal{O}$ is selected, the agent follows the option’s internal policy π_o until the termination condition β_o evaluates to *True*, after which the agent resumes selecting from the expanded action-option space.

3.2 Options as Actions in Abstract Graph

The state space $\mathcal{S}$ and the transition function $\mathcal{T}$ define, respectively, the nodes and edges of a directed graph. In order to obtain an abstract, simplified representation of this graph, we introduce the notion of Concept Function.

Definition 3.2. (Concept Function). A concept function is a function denoted by $\phi: \mathcal{S} \rightarrow \mathcal{S}^\phi$, where $\mathcal{S}^\phi$ is a connected partition of the state space $\mathcal{S}$, that is, the induced sub-graphs are connected graphs.

A Concept Function should capture salient environmental attributes in a coarse, abstract representation $\mathcal{S}^\phi$. We insist that the partitions induced by concept functions must

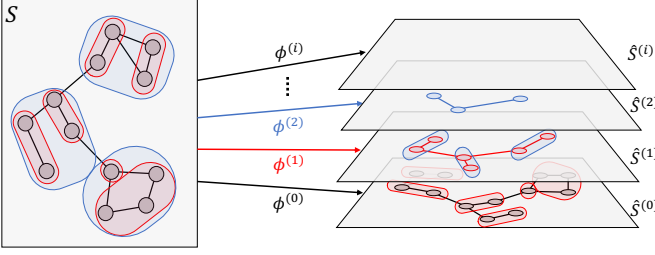


Fig. 1. Progressive compression of the environment graph given a hierarchical abstraction.

be connected. This prerequisite is vital for eliminating ambiguity in an agent’s movements within the environment. In this study, we presume that prior knowledge allows the derivation of these concepts, enabling their direct manual definition.

In an MDP, an action a is represented by the transition dynamics through the transition function $\mathcal{T}_a$. With the same objective in mind, we define an abstract action as a^ϕ to be represented by a transition function $\mathcal{T}_{a^\phi}: \mathcal{S}^\phi \rightarrow \mathbb{P}(\mathcal{S}^\phi)$. The abstract action a^ϕ represents the target transition dynamic that the agent aims to achieve in the abstract state space $\mathcal{S}^\phi$.

Similarly to actions, options can be associated with an empirical transition function $\mathcal{T}_o(s, s')$, which describes the probability that option o ends in state s' given that it was initiated in state s . The multi-step trajectory performed by an option o can be interpreted as one-step in the abstract state space $\mathcal{S}^\phi$.

Consequently, we establish a one-to-one correspondence between abstract actions and options, denoting each option characterized by a specific abstract action a^ϕ as o_{a^ϕ} . For this purpose, we will formalize the aim of options as minimizing the dissimilarity between the target transition $\mathcal{T}_{a^\phi}$ and the empirical transition $\mathcal{T}_{o_{a^\phi}}$.

A single concept function can be associated with multiple actions, discerning the abstract action space

$$\mathcal{A}^\phi = \{a_1^\phi, a_2^\phi, \dots, a_k^\phi\}.$$

This, in turn, defines the corresponding option space¹

$$\mathcal{O}^\phi = \{o_{a_1^\phi}, o_{a_2^\phi}, \dots, o_{a_k^\phi}\}.$$

A Concept Function ϕ can also be used to naturally define a termination condition β^ϕ , which triggers when the concept changes, i.e., when the agent transitions from one abstract state to another. This termination condition is shared across all options in $\mathcal{O}^\phi$ and can be formally defined as:

Definition 3.3. (Concept Termination condition). Given a concept function ϕ , the termination condition β^ϕ is defined as:

$$\beta^\phi(s, a, s') = \begin{cases} \text{True} & \text{if } \phi(s) \neq \phi(s') \\ \text{False} & \text{otherwise} \end{cases} \quad (3)$$

¹ To simplify the discussion, we only consider the case where the number of options is the same for all concepts, but in principle, each concept could be associated with a different number of options.

3.3 Structured Options using Hierarchical Abstractions

Traditionally, options are defined directly in terms of atomic environment actions. However, allowing options to be defined in terms of other options can lead to a more structured and efficient learning process. In our framework, we organize options in a multi-layer hierarchy, in order to obtain a structured and well-behaved option dependency graph.

Definition 3.4. (Hierarchical Abstraction). A set of concept functions $\phi_0, \phi_1, \dots, \phi_n$ defines a hierarchical abstraction if the respective state space partitions $\mathcal{S}^{\phi_i}$ are progressive refinements of one another. That is, $\mathcal{S}^{\phi_i} \supset \mathcal{S}^{\phi_{i+1}}$ for all $i \in (0, 1, \dots, n-1)$.

A concept function that is part of a hierarchical abstraction will be denoted as an Abstraction Function.

The refinement relation is a partial order in the set of all partitions, therefore not all combination of concept functions can be organized as hierarchical abstractions.

The identity function is the most trivial example of a concept function, where the state space is partitioned into singletons. This is useful for defining the base layer of the hierarchy, where the agent can perform atomic actions in the environment. Therefore, we define the base layer of the hierarchy ϕ_0 to be the identity, meaning that $\mathcal{S}^{\phi_0} = \mathcal{S}$.

It is also convenient to slightly modify the shared termination condition β^{ϕ_0} to be always true, instead of triggering only when the state changes. This way, options at this layer of abstraction are directly connected to atomic actions in the environment.

By requiring that partitions induced by abstraction functions are refinements of one another, we ensure that edges in higher-order graphs can be decomposed into multiple movements between lower-level states.

This allows options at a higher abstraction level i can be defined in terms of options at lower levels $j < i$. That is, the policy of a high-level option can invoke lower-level options as its actions, resulting in a nested, hierarchical structure. This recursive composition enables complex behaviors at higher layers to be constructed from simpler, reusable subroutines at lower layers, facilitating efficient learning and transfer across tasks.

Definition 3.5. (Intra-layer Policy). An *intra-layer policy* for the option $o \in \mathcal{O}^{\phi_i}$ is formally defined as

$$\hat{\pi}_{o^{\phi_i}}: \mathcal{S} \rightarrow \mathbb{P}(\mathcal{O}^{\phi_{i-1}})$$

and defines how options at the lower layer of abstraction can be combined to obtain the desired behaviour.

The intra-layer policy $\hat{\pi}_{o^{\phi_i}}$ implicitly defines routine associated to the option o^{ϕ_i} , just like the agent policy in 2 is used to express complex agent behavior in terms of simpler options. The option routine is obtained by selecting an option in $\mathcal{O}^{\phi_{i-1}}$. This lower-level option is followed until the termination condition $\beta_t^{\phi_{i-1}}$ is achieved, i.e., a state transition is executed at the lower abstraction layer $\mathcal{S}^{\phi_{i-1}}$. After the transition is completed, once again the policy $\hat{\pi}_{o^{\phi_i}}$ provides a probability distribution that leads to the selection of the next option.

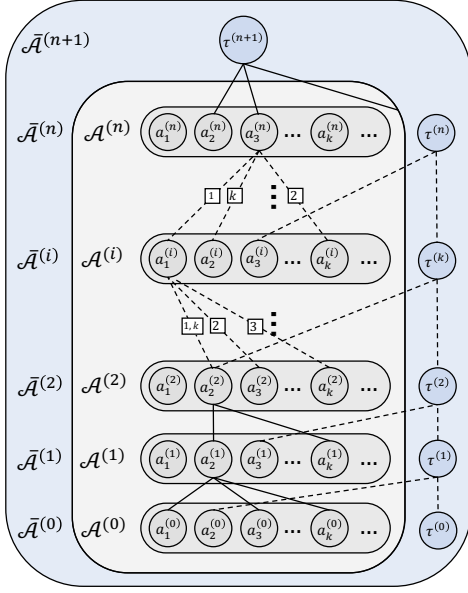


Fig. 2. Schematic representation of the abstract action sets and their interconnection, including the task options at each level of abstraction τ^{ϕ_i} .

3.4 Task actions

Solving a particular task in hierarchical RL amounts to maximizing the expected return in Equation 1 accumulated following the agent policy μ .

Unfortunately, given n layers of abstractions $\phi_0, \phi_1, \dots, \phi_n$, each with option set of size $|\mathcal{O}^{\phi_i}| = k$, yields a combined options set of size $(n+1) \cdot k$. Thus, the size of the option set obtained using the proposed framework grows relatively rapidly as more and more abstraction layers are considered, making the direct optimization of the policy μ increasingly difficult. However, the agent policy μ can be restructured in a way that takes advantage of the hierarchical organization of the option space: instead of a single policy over a large option space, a hierarchy of simpler policies is used.

Similarly to the hierarchical organization of options, the agent's policy can be arranged within the hierarchy as an additional abstract action integrated to each layer with an additional instantiation of an upper layer $(n+1)$ that will move the agent using the highest level options.

Definition 3.6. (Task Actions and Policies). A *task action* τ^{ϕ_i} is an abstract action defined at any layer $i \in \{0, \dots, n, n+1\}$. The corresponding expanded abstract action space is given by $\bar{\mathcal{A}}^{\phi_i} := \mathcal{A}^{\phi_i} \cup \{\tau^{\phi_i}\}$.

Each task action τ^{ϕ_i} induces a corresponding *task policy*, denoted by $\pi_{\tau^{\phi_i}} : \mathcal{S} \rightarrow \mathbb{P}(\bar{\mathcal{A}}^{\phi_{i-1}})$, which operates over the expanded action space of the layer below. These policies, analogous to those associated with other abstract actions, capture the hierarchical structure of decision-making. Due to the temporal abstraction introduced by this recursive structure, task policies are defined as *semi intra-layer policies* $\hat{\pi}_{\tau^{\phi_i}}$.

While task actions mirror abstract actions structurally, they differ in termination: abstract actions end on an abstract state change, whereas task options terminate

without inducing any state change at layer ϕ_i . Execution cascades upward, allowing higher-layer intra-layer policies to resume execution, and terminates upward, layer by layer, until reaching the top-level option.

The resultant subtrajectories are then linked either to previously trained options characterized by abstract actions $a^{\phi_{i-1}}$ or to task options $o_{\tau^{\phi_i}}$ if identified by $\tau^{\phi_{i-1}}$. The termination function.

Definition 3.7. (Task option termination function). Given the task option τ^{ϕ_i} , the termination function $\beta^{\tau^{\phi_i}}$ is defined as:

$$\beta_{\tau^{\phi_i}}(s_t, \bar{a}_t^{\phi_{i-1}}, s_t) = \begin{cases} \text{True if } \bar{a}_t^{\phi_{i-1}} = \tau^{\phi_{i-1}} \\ 0 \text{ otherwise} \end{cases}$$

except for $o_{\tau^{(0)}}$, which is the null option (empty routine)

3.5 Learning Process

We defined a generic option $o_{\bar{a}^{\phi_i}}$ associated with the action $\bar{a}^{\phi_i} \in \bar{\mathcal{A}}^{\phi_i}$ as a routine in the base layer, where $\bar{a}^{\phi_i}$ is an abstract action or a task action. This option routine comprises a chain of semi-intra-layer policies, which navigate through the states of the (j) -th level of abstraction with $j < i$ until the termination conditions are encountered.

An MDP in the layer of abstraction ϕ_i consists of the set of states $\mathcal{S}$ introduced in Section 2, the abstract actions $\mathcal{A}^{\phi_i}$ introduced in Section 3.1, the group of transition function $\mathcal{T}_{a^{\phi_i}}$ with $a^{\phi_i} \in \mathcal{A}^{\phi_i}$ introduced in Section 3.1 and a specific discount factor $\gamma\phi_i$. Given that the MDP provide the structure to train the specific policy that replicates the trajectory within layer ϕ_i when executing an action $a^{\phi_{i+1}}$ from the layer above, we characterize the reward associated to this specific action defining it by $R_{a^{\phi_{i+1}}}$. Since there are multiple actions from the layer above $a^{\phi_{i+1}} \in \mathcal{A}^{\phi_{i+1}}$, corresponding to different options $o_{a^{\phi_{i+1}}}$ with their intra-layer policies in layer ϕ_i , we create distinct MDPs within layer ϕ_i with identical structural elements $(\mathcal{S}, \bar{\mathcal{A}}^{\phi_i}, \{\mathcal{T}_{\bar{a}^{\phi_i}} | \bar{a}^{\phi_i} \in \bar{\mathcal{A}}^{\phi_i}\}, \gamma\phi_i, R_{\bar{a}^{\phi_{i+1}}})$ but varying reward functions aligned with the abstract actions from the layer above.

Definition 3.8. (SMDP of the action a^{ϕ_i}). Given the two Layer of a Hierarchical Abstraction ϕ_i and ϕ_{i+1} , given an action $\bar{a}^{\phi_{i+1}} \in \bar{\mathcal{A}}^{\phi_{i+1}}$ we define the Markov Decision Process of that action as the tuple of: structural objects, the discount factor and the reward functions that shape the environment of the layer ϕ_i according to given action $\bar{a}^{\phi_{i+1}}$, $(\mathcal{S}, \bar{\mathcal{A}}^{\phi_i}, \{\mathcal{T}_{\bar{a}^{\phi_i}} | \bar{a}^{\phi_i} \in \bar{\mathcal{A}}^{\phi_i}\}, \gamma\phi_i, R_{\bar{a}^{\phi_{i+1}}})$. $R_{\bar{a}^{\phi_{i+1}}} : \mathcal{S} \times \bar{\mathcal{A}}^{\phi_i} \times \mathcal{S} \rightarrow \mathbb{R}$ is the reward function such that $R_{\bar{a}^{\phi_{i+1}}}(s_t, a_t^{\phi_i}, s_{t+1})$ is given by:

$$\sum_{(s_k, a_k, s_{k+1}) \sim o_{\bar{a}^{\phi_{i+1}}}} \gamma_{\phi_i}^k r_{\bar{a}^{\phi_{i+1}}}(s_k, a_k, s_{k+1}) \quad (4)$$

and $r_{\bar{a}^{\phi_{i+1}}}(s_k, a_k, s_{k+1})$ is defined as:

$$\begin{cases} R(s_k, a_k, s_{k+1}) & \text{if } \bar{a}^{\phi_{i+1}} = \tau^{\phi_{i+1}} \\ \beta_k^{\phi_{i+1}}(1 - 2d_{\tau_{a^{\phi_{i+1}}}}(\delta(\phi^{\phi_i}(s_k), \phi^{\phi_i}(s_{k-1})))) & \text{if } \bar{a}^{\phi_{i+1}} = a^{\phi_{i+1}} \end{cases} \quad (5)$$

The function $r_{\bar{a}^{\phi_{i+1}}}(s_k, a_k, s_{k+1})$ takes into consideration the current state s_k , the action a_k taken, and the subse-

quent state s_{k+1} , all contingent on the macro action $\bar{a}^{\phi_{i+1}}$ at hierarchy level $i+1$. When $\bar{a}^{\phi_{i+1}}$ corresponds to a task action $\tau^{\phi_{i+1}}$, the reward is derived from the standard reward function $R(s_k, a_k, s_{k+1})$. However, if $\bar{a}^{\phi_{i+1}}$ represents an abstract action $a^{\phi_{i+1}}$, the reward is intrinsic in the structure and influenced by the abstract transition generated by the trajectory. It is nonzero only if the trajectory terminates, which is indicated by $\beta_{k+1}^{\phi_{i+1}} = 1$. Additionally, the reward is adjusted based on the alignment of the occurred abstract transition with the intended transition defined by the abstract action dynamics, quantified by $(1 - 2d\tau_{a^{\phi_{i+1}}}(\delta(\phi^{\phi_i}(s_k), \phi^{\phi_i}(s_k - 1))))$.

It is then possible to explicitly write the Q-function related to the training of the policy of the an option at a given layer ϕ_i as $Q_{\bar{a}^{\phi_i}}(s_t, o_{\bar{a}_t^{\phi_{i-1}}})$ where $\bar{a}_t^{\phi_{i-1}} \in \bar{\mathcal{A}}^{\phi_{i-1}}$:

$$Q_{\bar{a}^{\phi_i}}(s_t, \bar{a}_t^{\phi_{i-1}}) = \mathbb{E}_{s_{t+1} \sim \pi_{o_{\bar{a}_t^{\phi_{i-1}}}}} [r_{\bar{a}^{\phi_i}}(s_t, \bar{a}_t^{\phi_{i-1}}, s_{t+1}) + \gamma^{\phi_{i-1}} \cdot \max_{\bar{a}_{t+1}^{\phi_{i-1}} \in \bar{\mathcal{A}}^{\phi_{i-1}}} Q(s_{t+1}, \bar{a}_{t+1}^{\phi_{i-1}})] \quad (6)$$

Here, $Q_{\bar{a}^{\phi_i}}(s_t, \bar{a}_t^{\phi_{i-1}})$ represents the expected cumulative reward gained by the policy $\bar{a}^{\phi_i}$ starting from state s_t and executing the option characterized by the action $\bar{a}_t^{\phi_{i-1}}$. The expectation is taken over the state distribution s_{t+1} at the end of the policy characterized by the action $\bar{a}_t^{\phi_{i-1}}$. The expression $r_{\bar{a}^{\phi_i}}(s_t, o_{\bar{a}_t^{\phi_{i-1}}}, s_{t+1})$ denotes the reward shaped by $\bar{a}^{\phi_i}$ obtained by executing option $o_{\bar{a}_t^{\phi_{i-1}}}$ at state s_t , and $\gamma^{\phi_{i-1}} \cdot \max_{\bar{a}_{t+1}^{\phi_{i-1}} \in \bar{\mathcal{A}}^{\phi_{i-1}}} Q(s_{t+1}, \bar{a}_{t+1}^{\phi_{i-1}})$ is the discounted maximum expected future reward from the next state s_{t+1} , considering the abstract actions available for the subsequent step $\bar{\mathcal{A}}^{\phi_i}$.

After training various task actions at each layer, the top-level task option $\tau^{\phi_{i+1}}$ can then act as the agent’s policy, represented as μ . This policy oversees the entire process, utilizing the options to orchestrate the agent’s movements and terminates with the conclusion of the environment’s entire process.

4. EXPERIMENTAL SETUP AND RESULTS

4.1 Environment Description

We adopt a grid-based environment loosely inspired by Frozen Lake, where each cell can be frozen (safe to walk on) or a hole (causing an immediate episode termination). The agent starts in one cell, and the goal (or “gift”) is placed in another. The agent earns a reward upon reaching this goal, and each episode terminates either when the agent succeeds (reaching the goal) or fails (falling into a hole or exceeding a step limit). Once generated, the map remains fixed, but the agent and the goal object (“gift”) are placed randomly at each reset.

To apply MANGO, we define a hierarchy of abstraction functions ϕ_i that compress contiguous regions of the grid into “supercells.” For instance, at level i , each $(2^i \times 2^i)$ -block of the original map collapses into a single abstract cell, preserving only the essential one-hot information for

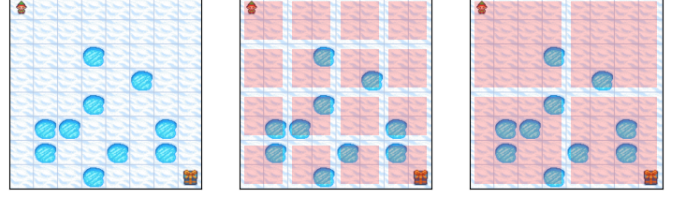


Fig. 3. Abstraction functions applied to the Frozen Lake environment

the agent’s position and goal. By ensuring connectedness and consistent transitions, these abstractions meet the criteria in Section 3.2.

At each layer i , we define the movement actions in the compressed (abstract) grid $\{\text{UP}, \text{LEFT}, \text{DOWN}, \text{RIGHT}\}$. An abstract action thus corresponds to relocating the agent from one abstract cell to an adjacent one at layer i , with collisions or holes folded into that layer’s transition dynamics.

Figure 3 illustrates how blocks of the map are pooled at each layer, visually conveying which states are aggregated and how transitions map between adjacent abstract states.

4.2 Training Procedure

A set of parallel workers then collects environment rollouts by interacting with the map. These transitions, including states, rewards, and terminations, serve as input for off-policy RL updates.

The first (lowest) layer of MANGO trains for a prescribed number of steps, refining a policy that learns basic navigation and the ability to avoid holes. When this layer’s budget is exhausted or its performance is deemed satisfactory, its parameters are frozen, meaning that future layers can treat it as part of the environment’s transition model of the upper layer. The next layer operates on a more abstract representation of the map, delegating detailed actions to the frozen sub-policy. This process continues until all hierarchical layers have undergone training, each refining a policy at its own level of abstraction.

Multiple objective functions are tested to balance early improvements against ultimate convergence. These include maximizing the area under the reward-step curve, combining final accuracy with training length, and rewarding both fast and stable convergence. In practice, the method proves highly sensitive: if the first layer is not adequately trained, its errors propagate to subsequent layers, preventing the outer policy from achieving full accuracy. Conversely, overly aggressive learning rates can produce rapid initial gains but hamper fine-tuning.

4.3 Results

The hierarchical training strategy described in Section 4.2 can successfully guide the agent on 16×16 randomly generated Frozen Lake maps, provided the hyperparameter configuration is carefully tuned. In many trials, the agent achieves near-perfect completion rates once each layer’s policy has been given enough training steps to master its designated abstraction as shown in Figure 4. However, even slight misconfigurations in learning rates, soft-update

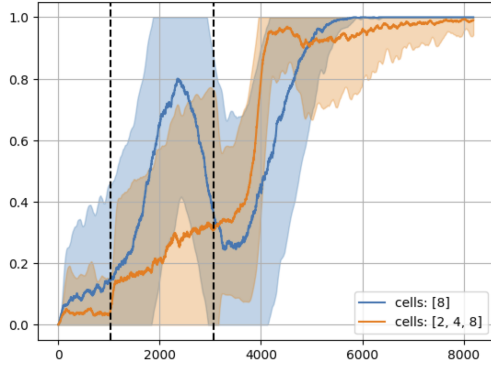


Fig. 4. Convergence of MANGO and Q-learning with hyperparameter optimization, the vertical dashed lines represent when MANGO starts the training of the upper layer. We can observe that mango reaches the first high percentage but saturates later than Q-learning.

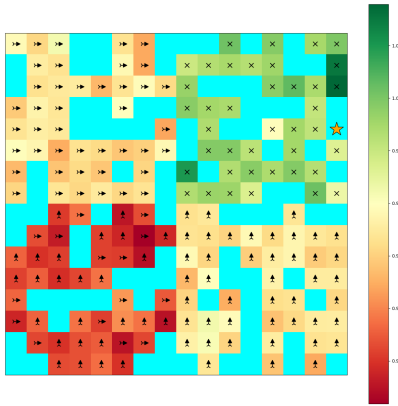


Fig. 5. Q-values of the overarching task action at level 4 of abstraction.

schedules, or layer-wise training budgets often lead to partial convergence or complete failure.

One consistent observation is that if the base layer does not thoroughly learn to navigate local obstacles and avoid holes, higher layers inherit an unreliable subroutine for their coarse abstractions, causing the entire policy stack to stall below optimal performance. In contrast, when the first layer converges robustly, subsequent layers can assemble its transitions into longer, more abstract paths that accelerate exploration and convergence. Yet this hierarchical advantage vanishes under overly aggressive learning rates, which may yield rapid initial gains at lower layers but prevent fine-tuning; Optuna’s pruning can also prematurely end otherwise promising runs that start slowly.

5. CONCLUSION

This work introduced MANGO, a hierarchical reinforcement learning framework that decomposes complex environments into layers of abstract state transitions and nested options. Through this structure, MANGO improves sample efficiency and interpretability by enabling agents to reason and act at multiple levels of abstraction. Experiments in sparse-reward grid environments demonstrate the framework’s effectiveness when properly tuned, although

performance remains sensitive to the quality of lower-layer policies and training schedules.

Beyond improved learning efficiency, MANGO enhances interpretability by making the agent’s decision-making process transparent across layers—an asset in safety-critical or industrial settings. Future work will explore automated discovery of abstractions and abstract actions, adaptation to continuous or fuzzy environments, and more robust multi-layer training strategies. These directions aim to strengthen MANGO’s scalability, stability, and autonomy in increasingly complex tasks.

REFERENCES

- Abel, D. (2022). A theory of abstraction in reinforcement learning. In *arXiv preprint arXiv:2203.00397*.
- Abel, D., Umbanhowar, N., Khetarpal, K., Arumugam, D., Precup, D., and Littman, M.L. (2020). Value preserving state-action abstractions. In *International Conference on Artificial Intelligence and Statistics*.
- Dietterich, T.G. (1999). Hierarchical reinforcement learning with the maxq value function decomposition. *ArXiv*, cs.LG/9905014.
- Hengst, B. et al. (2002). Discovering hierarchy in reinforcement learning with hexq. In *Icml*, volume 19, 243–250. Citeseer.
- Ho, M.K., Abel, D., Correa, C.G., Littman, M.L., Cohen, J.D., and Griffiths, T.L. (2021). People construct simplified mental representations to plan. *Nature*, 606, 129 – 136.
- Ho, M.K., Abel, D., Griffiths, T.L., and Littman, M.L. (2019). The value of abstraction. *Current Opinion in Behavioral Sciences*, 29, 111–116.
- Icarte, R.T., Klassen, T.Q., Valenzano, R., and McIlraith, S.A. (2022). Reward machines: Exploiting reward function structure in reinforcement learning. *Journal of Artificial Intelligence Research*, 73, 173–208.
- Konidaris, G.D. and Barto, A.G. (2009). Efficient skill learning using abstraction selection. In *International Joint Conference on Artificial Intelligence*.
- Konidaris, G.D., Kaelbling, L.P., and Lozano-Perez, T. (2015). Symbol acquisition for probabilistic high-level planning. In *International Joint Conference on Artificial Intelligence*.
- Rafati, J. and Noelle, D.C. (2018). Learning representations in model-free hierarchical reinforcement learning. *ArXiv*, abs/1810.10096.
- Ravindran, B. and Barto, A.G. (2003). Smdp homomorphisms: An algebraic approach to abstraction in semi-markov decision processes. In *International Joint Conference on Artificial Intelligence*.
- Sacerdoti, E.D. (1974). Planning in a hierarchy of abstraction spaces. *Artif. Intell.*, 5, 115–135.
- Silver, D. and Ciosek, K. (2012). Compositional planning using optimal option models. In *ICML*.
- Sutton, R.S., Precup, D., and Singh, S. (1999). Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2), 181–211.
- Veeriah, V., Zahavy, T., Hessel, M., Xu, Z., Oh, J., Kemaev, I., Hasselt, H.V., Silver, D., and Singh, S. (2021). Discovery of options via meta-learned subgoals. In *Neural Information Processing Systems*.